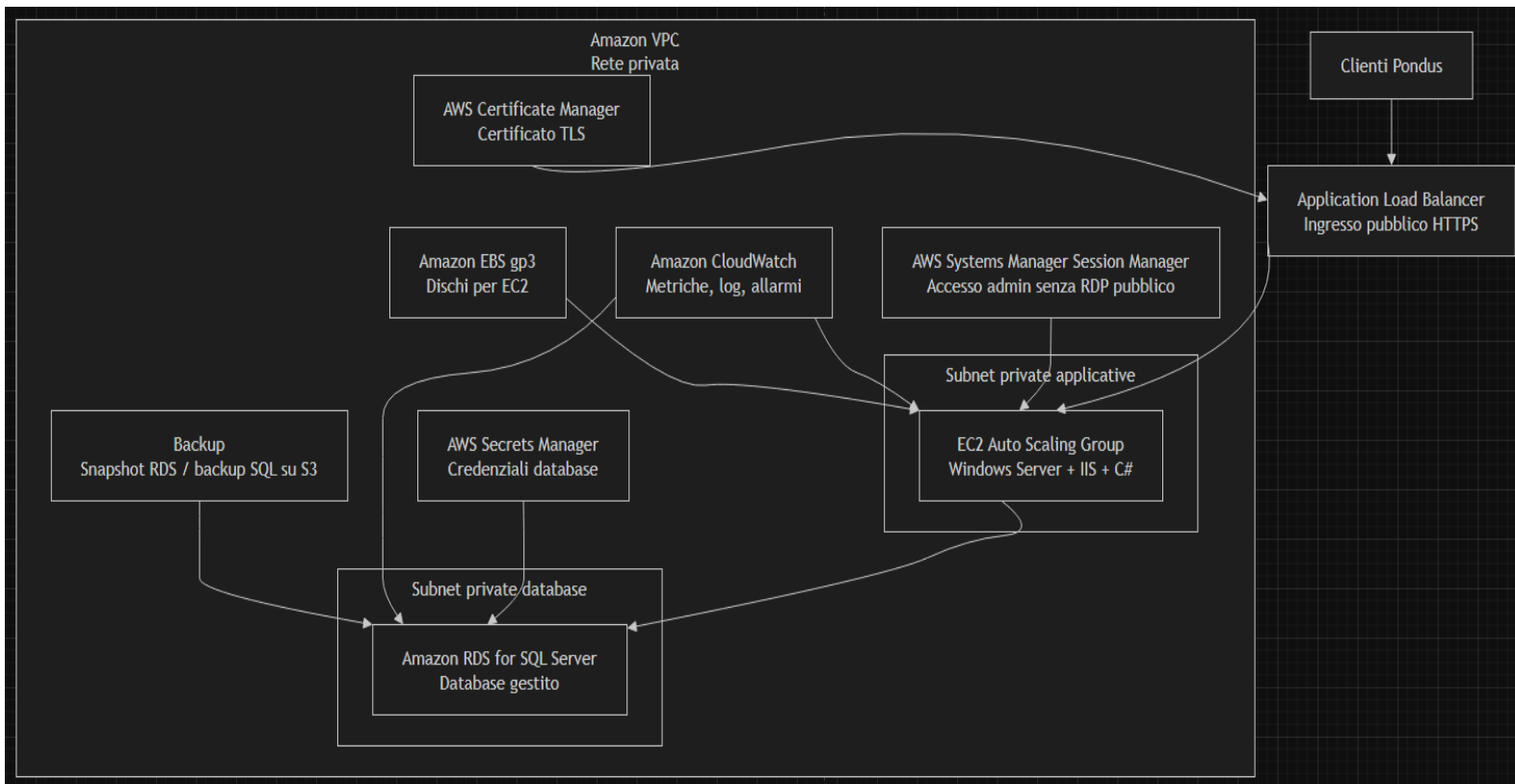


FIRST TRY



Architettura semplice

ALB + poche istanze Windows EC2 per IIS e RDS for SQL Server se la licenza lo consente;

in alternativa, SQL Server su EC2 solo se il vincolo di licenza blocca RDS.

Struttura consigliata

Eviterei un autoscaling aggressivo: meglio un setup con capacità minima stabile, ad esempio 1–2 EC2 Windows dietro Application Load Balancer, perché l'ALB distribuisce il traffico sulle istanze registrate e può lavorare insieme ad Auto Scaling per tenere il numero di server sotto controllo.

Questo si adatta bene a un'app grande ma con utenza limitata, dove la priorità è la continuità del servizio più che la crescita elastica. (dell'app per se)

Architettura per Pongus

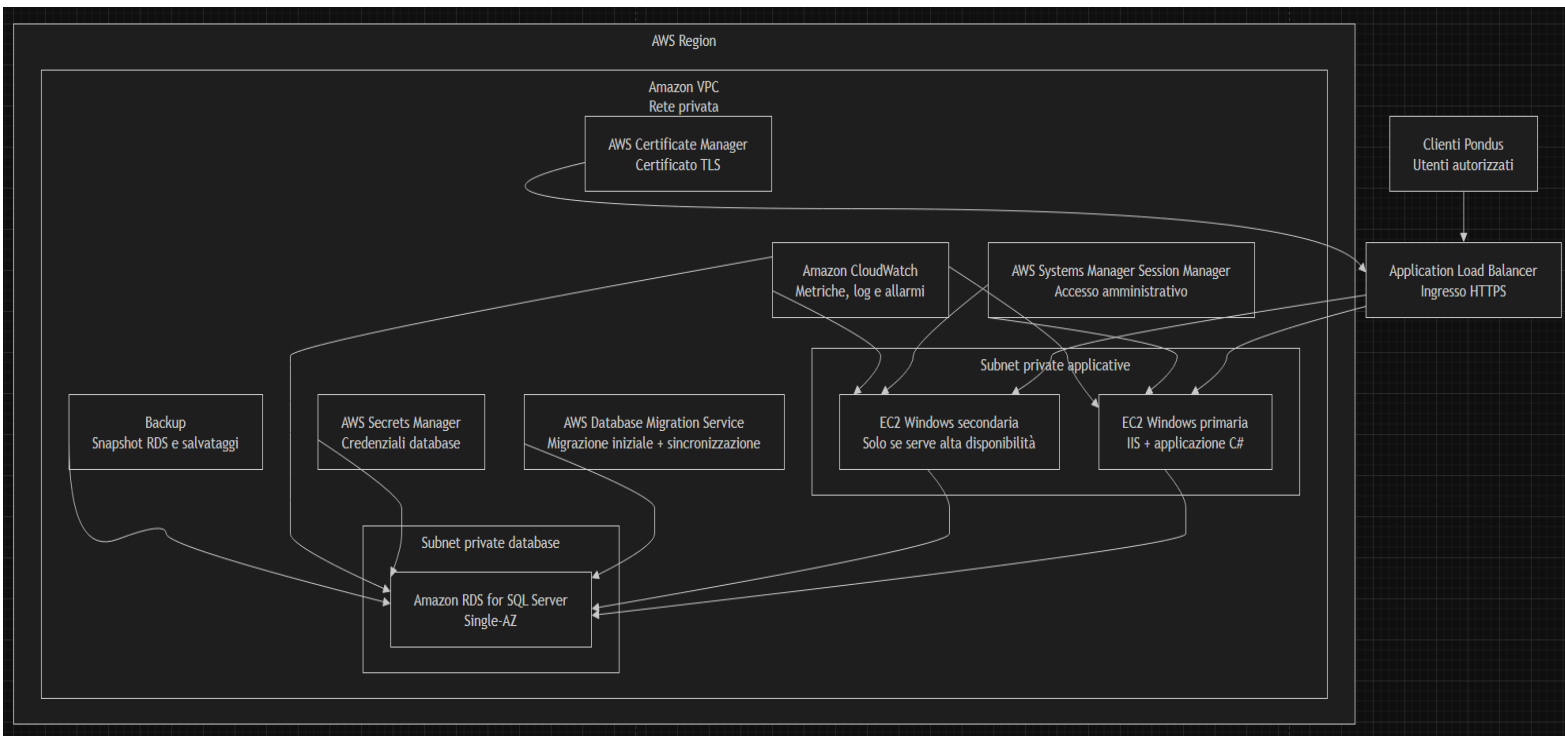
- **Frontend/Web**: Application Load Balancer pubblico + EC2 Windows con IIS e applicazione C# in subnet private .
- **Database**: Amazon RDS for SQL Server single-AZ per partire in modo più economico; Multi-AZ solo se il budget lo consente e il downtime è davvero critico.

- **Storage:** EBS gp3 per le istanze EC2, mentre lo storage del DB è gestito da RDS se usi RDS.
- **Accesso admin:** SSM Session Manager al posto di NAT Gateway e RDP esposto, per ridurre costi e superficie d'attacco.
- **Segreti e TLS:** Secrets Manager per le credenziali, ACM per il certificato TLS dell'ALB.

Scelte chiave

Per la migrazione database, AWS DMS è adatto se vuoi **replica continua e downtime minimo**: AWS documenta l'uso di "replicate ongoing changes" per mantenere sincronizzati source e target durante la migrazione. Per il dimensionamento iniziale, conviene partire con un profilo "right-sized" e aumentare solo se i test lo richiedono; RDS per SQL Server include licenza e può essere acquistato on-demand o con Reserved Instances/Savings Plans. Se il carico resta basso, una strategia a scalabilità limitata evita costi inutili e resta più facile da gestire.

SECOND TRY



L'architettura di Pondus su AWS è pensata per un'app grande ma con pochi clienti, quindi con una scalabilità limitata e costi sotto controllo. La soluzione prevede un **Application Load Balancer** come punto di ingresso pubblico, alcune **EC2 Windows** con IIS per l'app C#, e **Amazon RDS for SQL Server** se la licenza lo permette. [aws.amazon+1](#)

Il database viene migrato con **AWS DMS**, così da ridurre il downtime e mantenere sincronizzati sorgente e destinazione durante il passaggio. L'accesso amministrativo avviene con **Systems Manager Session Manager**, che evita di esporre RDP su Internet e semplifica la gestione delle istanze. [aws.amazon+2](#)

Le credenziali sono salvate in **Secrets Manager**, mentre il certificato TLS è gestito con **ACM**. Per monitoraggio e backup si usano **CloudWatch** e snapshot RDS, così da avere controllo operativo e possibilità di ripristino.

Se la licenza RDS non è utilizzabile, il database può restare su **SQL Server in EC2**, ma questa opzione richiede più gestione e comporta maggiore complessità operativa.